

zod x TestPilot: All 124 Functions and Coverage

This week we repeated last week's experiment on all 124 user-callable functions of zod's main entry point instead of a 60-function sample, with everything else unchanged. We also measured statement and branch coverage as in the TestPilot paper, next to zod's own developer tests. Date: 2026-09-25.

1. Questions

1. With 124 functions instead of 60 and nothing else changed, how do generation and survival compare with last week?
2. What coverage do the passing generated tests reach, measured as in the paper (RQ1)?
3. As context, what does the developer suite cover, and which of the code later releases changed did each corpus execute?

2. Cases

- **Subject:** zod at t = v4.0.5, later points the six minor releases v4.1.0 to v4.6.0 (as last week).
- **Functions:** all 124 functions of the main entry (z.<name>, z.coerce.*, z.iso.*; 74 schema factories, 35 checks and refinements, 15 parsing, coercion, format and configuration functions). Last week's 60 are a subset.
- **Generation:** unchanged from last week. testpilot2, model gpt-oss-120b via OpenRouter, temperature 0, one completion per prompt, the same prompt settings and the same 20 documentation files. The passing set S124 is all tests passing at t.
- **Developer corpus:** zod's 81 test files (888 cases) at v4.0.5, frozen.

3. Protocol

- **Generation and survival:** exactly as last week. Each generated test runs alone under Mocha against the built package at each release. The developer suite from v4.0.5 runs under each release's vitest.
- **Coverage:** measured at t over a fixed file set D, the 63 TypeScript files that require('zod') loads (8 in the public API layer classic, 14 in core, 40 locale message files, one index).
 - Generated tests are measured with nyc, as testpilot2's own validator does. They run on a copy of v4.0.5 rebuilt with source maps, whose JavaScript is byte-identical to the original apart from the source-map comment, so coverage is reported on the TypeScript source.
 - The developer suite is measured with vitest's Istanbul provider.
 - The two routes count statements differently, so the two corpora are compared at line level.

4. Results

Generation at t.

	Last week (60 functions)	This week (124 functions)
Tests / passing	360 / 139 (38.6%)	757 / 266 (35.1%)
Functions with a passing test	54	112

On the 60 shared functions, 232 of 236 non-retry prompts were identical to last week's, but only 1 of 373 generated test files was identical. At temperature 0 the model gave different answers week to week.

Survival (control at v4.0.5: 266/266 and 888/888):

Release	Generated tests passing / 266	Developer cases passing / 888
v4.1.0	265 (99.6%)	871 (98.1%)
v4.2.0	266 (100.0%)	868 (97.7%)
v4.3.0	263 (98.9%)	864 (97.3%)
v4.4.0	263 (98.9%)	853 (96.1%)
v4.5.0	261 (98.1%)	838 (94.4%)
v4.6.0	258 (97.0%)	834 (93.9%)

The developer results are identical to last week's at every release, case by case. Nine generated tests fail at some release, all in functions outside last week's sample:

- three instanceof tests from v4.3.0 (default error message);
- two partialRecord tests from v4.5.0 (error-message regular expression);
- one partialRecord test at v4.1.0 only, which passes again from v4.2.0;
- iso.date, emoji and toUpperCase at v4.6.0 (the iso.date and toUpperCase tests replace zod.z with their own stub; see Observations).

Coverage, paper metrics (passing tests; all tests still pass under nyc):

Over D	Loading only	S124 (266 tests)	S60 (last week's 139)	Developer suite (reference)
Statements	15.2%	46.4%	39.2%	59.7%
Branches	0.2%	20.0%	15.4%	35.4%

- Uniquely contributing tests (covering a statement no other test covers): 16.2% of S124, 20.1% of S60.
- Per-function statement coverage has median 100% (17 of 124 functions at 0%), because most public functions are one-statement wrappers around core.
- For reference, the paper reports median statement coverage 70.2%, branch 52.8% and uniquely contributing tests 10.5%, on 25 other packages with a different model and tool version.

Line level against the developer suite (5070 executable lines):

Area	Lines	S124	Developer suite
classic (public API layer)	740	92.2%	75.3%
core (schemas, checks, parsing)	2462	61.4%	81.2%
locales (40 message tables)	1864	10.7%	8.9%
All of D	5070	47.3%	53.7%

Code changed by later releases. Of the 5070 executable lines of v4.0.5, 1946 (38%) were modified or deleted by v4.6.0. At t, S124 executed 727 of them and the developer suite 1221. 265 of the 266 S124 tests executed at least one of these lines, and the remaining one never loads zod. 257 of the 265 survived v4.6.0.

5. Observations

1. **Last week's near-zero breakage of generated tests depended partly on the sample.** With all 124 functions, 3.0% of the generated tests fail at v4.6.0, against 6.0% of the developer cases. All the generated-test failures are in functions the 60-function sample did not include.
2. **Most breaks of generated tests are not caused by a change in the tested behaviour.** Reading the nine failing tests gives three groups:
 - Five assert on error-message text that changed, like most developer failures (snapshots).
 - Two replace zod.z with their own stub, so they never tested zod. They broke only because v4.6.0 changed how zod exports z.
 - Two reflect a real change: emoji stopped accepting digits-only strings at v4.6.0, and partialRecord crashed at v4.1.0 only.

So a generated test can pass at t without exercising the function it is named for.

3. **Generated tests cover the API layer more and the core less than the developer suite.** They cover 92% against 75% of classic and 61% against 81% of core. This fits the shape of the tests seen last week: short, basic parse behaviour through the public functions. Leaving out the locale tables, which every test barely touches, line coverage is 68.5% for S124 and 79.7% for the developer suite.
4. **Line-level change coverage does not separate the surviving tests.** 38% of the lines changed and almost every test runs through them (median 219 changed lines per test). So this cannot show whether a surviving test reached a behaviour change and missed it. That needs the lines changed by the commits that actually broke tests.
5. **A single generation run is one sample.** The same prompts gave almost entirely different tests week to week, so survival and coverage figures of one run carry run-to-run variation that has not been measured.

6. Limitations

- The OpenRouter provider is not pinned or recorded, and is the likely cause of the week-to-week variation.
- The 124-function generation ran in two parts with identical settings, because the tool's default five-hour limit stopped the first part after 109 functions. The provider was about twice as slow per function as last week.
- Coverage is measured at t only.
- D includes all 40 locale files.
- The paper's non-trivial-assertion coverage (RQ3) and per-refiner ablation (RQ5) were not measured.
- Failure categories are automatic. Failures have not yet been classified manually.
- The zod/mini entry point is not included.

7. Next steps (for discussion)

1. Classify the 9 generated and 53 developer failures manually (contract change or non-contract assertion).
2. Bisect the breaks to their causing commits, and redo the change coverage on the lines those commits changed.
3. Run repeated generations of the same configuration to measure run-to-run variation.
4. Pair generated tests with developer cases at case level for each function.
5. Compute per-function coverage that follows the call into core.

Repository: [Hamjoon/zod](#), branch [experiment/2026-09-week4-testpilot-zod](#), directory [experiments/testpilot-2026-09](#).